

ANEXO V

Documentación técnica.

*Diseño y desarrollo de una plataforma robótica para la
agilización de procesos en la hostelería*

Trabajo de Fin de Grado
Grado en Ingeniería Informática



VNiVERSiDAD
DSALAMANCA

CAMPUS DE EXCELENCIA INTERNACIONAL

Julio 2022

Autor

David Cruz García

Tutores

André Filipe Sales Mendes
Gabriel Villarrubia González
Juan Francisco de Paz Santana

Tabla de contenido

1 INTRODUCCIÓN	3
2 ESTRUCTURA COMPLETA DEL SISTEMA.....	4
2.1 SERVIDOR CLIENTE O FRONTEND	4
2.2. SERVIDOR API O BACKEND	5
2.3 ROBOT	5
3 ESPECIFICACIÓN DE LA DOCUMENTACIÓN.....	5
4 REFERENCIAS	6

1 INTRODUCCIÓN

Este documento está enfocado a proporcionar una explicación de la aplicación desde el punto de vista del programador que quiera comprender el código.

El proyecto está estructurado en 3 apartados principales:

- **Servidor Cliente o Frontend:** es el servidor que proporciona la web a la que pueden acceder los usuarios. Este servidor se comunica mediante una API REST con el *Servidor API o Backend* para obtener los datos necesarios, realizar acciones en ellos o comunicarse con el robot.
- **Servidor API o Backend:** servidor que ofrece una serie de servicios a través de una API enfocada al Frontend (patrón BFF). Este servidor se encargará de toda la parte lógica del proyecto, accediendo a la base de datos, comunicándose con el robot o devolviendo resultados al *Servidor Cliente o Frontend*.
- **Robot:** incluye el firmware del robot y la API expuesta para la comunicación con dicho robot. El firmware utilizado es *“Marlin”* mientras que para la comunicación con el firmware mediante una API se ha decidido utilizar el SSOO OctoPi para RaspberryPi, que incluye la API *“Octoprint API”* con la posibilidad de mandar comandos GCODE de *Marlin* para el movimiento del robot.

2 ESTRUCTURA COMPLETA DEL SISTEMA

En este apartado se explicarán más detalladamente cada componente del sistema.

2.1 SERVIDOR CLIENTE O FRONTEND

El código fuente del servidor Frontend desarrollado se encuentra dentro de la carpeta “*codigoFuente/client-fe*”.

Este servidor ha sido desarrollado con Node.js y React, por lo que la estructura del proyecto es la siguiente:

- **Carpeta *src/*:** contiene todo el código encargado de generar las diferentes páginas de la web. Contiene a su vez diferentes carpetas y archivos:
 - ***index.js*:** archivo inicial que carga toda la página web llamando al método render del DOM de react. Se encarga de cargar el componente App.js encargado de cargar toda la página
 - ***App.js***
 - ***routes.js*:** archivo donde se especifican las diferentes rutas accesibles por el sistema.
 - ***config.js*:** fichero con diferentes variables de comunicación necesarias para el correcto funcionamiento del servidor y la correcta conexión con el Backend.
 - ***assets/*:** directorio encargado de guardar imágenes y temas que se usan en la aplicación
 - ***components/ y context/*:** guardan los componentes y el contexto de la aplicación para la creación de nuevos componentes. Aquí se almacenarán los componentes de la plantilla utilizada.
 - ***layouts/*:** carpeta donde se almacenan de forma ordenada las distintas páginas a las que accederán los usuarios. Estas páginas han sido autodocumentadas utilizando la herramienta JSDoc, la cual crea la carpeta “***docs/***” donde se almacena esta documentación.
 - ***services/*:** contiene los controladores encargados de realizar las peticiones a la API y que serán llamados desde las diferentes páginas para realizar distintas acciones.
- **Carpeta *docs/*:** contiene la documentación autogenerada por JSDoc
- **Carpeta *public/*:** usada por React, contiene el HTML que usa React para poder operar, y otros elementos como el favicon.

2.2. SERVIDOR API O BACKEND

El código fuente del servidor Backend desarrollado se encuentra dentro de la carpeta “*codigoFuente/server-be*”.

Este servidor ha sido desarrollado con Node.js y Express, por lo que la estructura del proyecto es la siguiente:

- ***app.js y bin/www***: ficheros de arranque de express donde se cargará toda la lógica necesaria para la creación de la base de datos, preparación de routers, etc para el correcto funcionamiento del servidor.
- ***routes/***: contiene los diferentes routers que ofrecen las rutas de la API a la que se enviarán peticiones. Esta carpeta está autodocumentada usando JSDoc.
- ***services/***: son los controladores que procesarán las peticiones recibidas por la API y generarán una respuesta o una acción en el sistema. Esta carpeta está autodocumentada usando JSDoc.
- ***models/***: incluye los diferentes modelos y relaciones que utilizará la biblioteca Sequelize para trabajar con la base de datos.
- ***middleware/***: incluye algunos middleware encargados de realizar algunas tareas de filtrado para algunas peticiones.
- ***mailer/***: guarda toda la lógica necesaria para el envío de emails y las plantillas HTML usadas para los mismos.
- ***docs/***: contiene la documentación autogenerada por JSDoc

2.3 ROBOT

En cuanto al robot, realmente lo único incluible es el firmware. Se trata del firmware “Marlin”, muy utilizado en impresoras 3D y que ha sido modificado según las necesidades del robot.

Este firmware modificado ha sido incluido en la carpeta “*codigoFuente/barbotModifiedMarlin*”.

Para la web y API del robot, se ha utilizado el sistema operativo OctoPi en la RaspberryPI, ofreciendo directamente ambas.

3 ESPECIFICACIÓN DE LA DOCUMENTACIÓN

Para la generación de la documentación técnica se ha utilizado la herramienta **JSDoc**, la cual genera ficheros HTML con la documentación del código a partir de comentarios incluidos en el propio código. Esta documentación se encuentra en las carpetas “**docs/**” de ambos servidores.

4 REFERENCIAS

- Documentación de JSDoc (accedido el 15-06-2022)
 - <https://jsdoc.app/>